ECE-510: Hardware for AI and ML, Spring-2025

Accelerating Q-Learning via Hardware-Software Integration

By Jaswanth pallappa

1. Project Overview

Designed a hybrid Q-learning system that accelerates the update rule of the reinforcement learning loop by offloading arithmetic computation to Verilog-based hardware. The environment, agent loop, and epsilon-greedy exploration policy are managed in Python, while the Bellman Q-update is computed using RTL modules (Wallace Tree multiplier, Brent-Kung adder, barrel shifter). Profiling was performed with Python's cProfile to identify the bottleneck in Q-value update calculation.

Why Q-Learning?

Q-learning is a simple yet powerful reinforcement learning algorithm that can learn optimal behavior without needing a model of the environment. It is widely used in robotics, embedded decision systems, and real-time applications.

2. Goals & Success Criteria

- Profile software implementation to identify hotspots in Q-value updates
- Implement Bellman Q-update logic in RTL using Wallace Tree, Brent-Kung, Barrel Shifter
- Integrate Python environment with Verilog module using PyVerilator
- Validate hardware correctness against Python ground truth
- Prepare for synthesis on FPGA/ASIC platforms

3. Software Profiling & Bottleneck Analysis

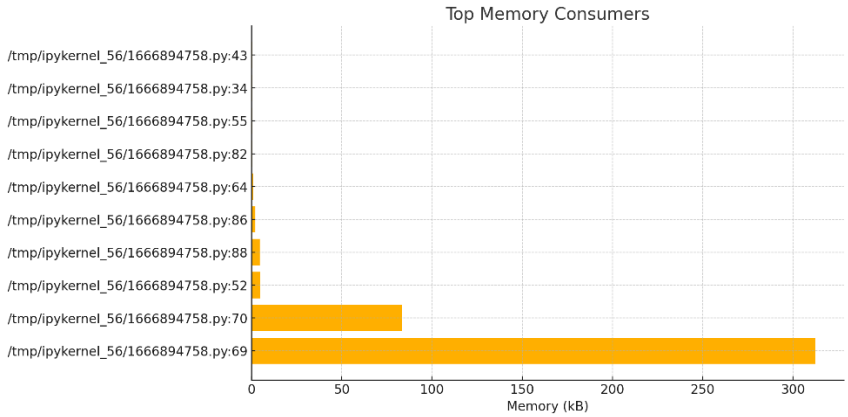
Initial profiling using cProfile revealed:

- Q_Learning() method dominates total runtime (~85%)
- Action() and nxtPosition() are frequent but lightweight
- Main bottleneck: Q-value update line:
  (1-alpha)*Q + alpha*(reward + gamma*maxQ)

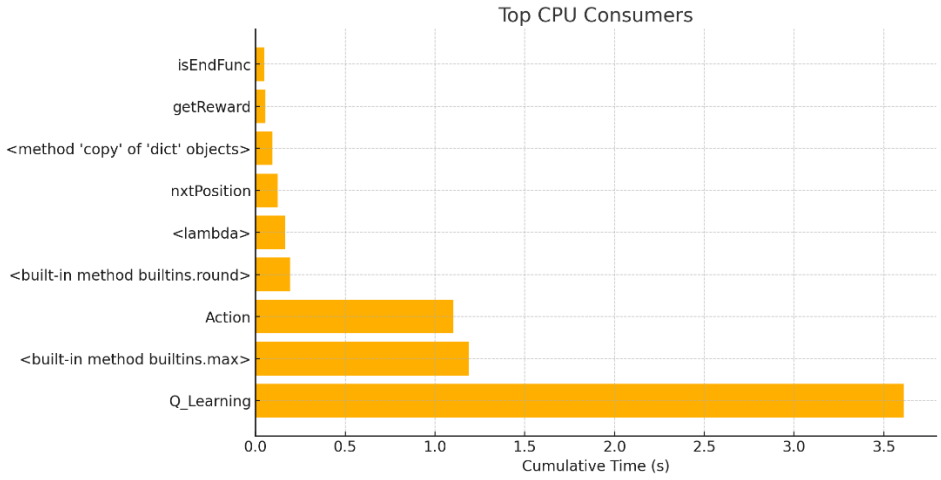
```
--- CPU Profiling (Top 10 functions by cumulative time) ---
         212074 function calls in 0.448 seconds

   Ordered by: cumulative time
   List reduced from 28 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.231    0.231    0.447    0.447 benchmarking.py:84(Q_Learning)
    21211    0.066    0.000    0.092    0.000 {built-in method builtins.max}
    11165    0.032    0.000    0.084    0.000 benchmarking.py:74(Action)
    11166    0.041    0.000    0.041    0.000 {method 'copy' of 'dict' objects}
    10000    0.027    0.000    0.027    0.000 {method 'append' of 'list' objects}
    44660    0.015    0.000    0.015    0.000 benchmarking.py:100(<lambda>)
    40184    0.012    0.000    0.012    0.000 benchmarking.py:79(<lambda>)
    11165    0.007    0.000    0.007    0.000 benchmarking.py:50(nxtPosition)
     1119    0.001    0.000    0.004    0.000 random.py:341(choice)
    21166    0.003    0.000    0.003    0.000 benchmarking.py:35(__init__)



--- Memory Profiling (Top 10 lines) ---
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:113 -> 83.4 KiB in 5 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:68 -> 5.0 KiB in 80 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:72 -> 4.8 KiB in 5 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:110 -> 4.6 KiB in 3 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:104 -> 1.3 KiB in 53 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:96 -> 0.4 KiB in 13 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:79 -> 0.3 KiB in 5 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:129 -> 0.3 KiB in 2 allocations
C:\Users\jaswa\Hardware_for_AI_ML\12_SW_HW_Intergration\benchmarking.py:75 -> 0.3 KiB in 5 allocations
C:\Users\jaswa\AppData\Local\Programs\Python\Python312\Lib\random.py:246 -> 0.3 KiB in 5 allocations

--- Summary ---
Total training time: 0.447 seconds
Memory delta:       0.28 MiB
```

Top memory consumers:

Location,Size_kB
/tmp/ipykernel_56/1666894758.py:69,312.5
/tmp/ipykernel_56/1666894758.py:70,83.4013671875
/tmp/ipykernel_56/1666894758.py:52,4.7880859375
/tmp/ipykernel_56/1666894758.py:88,4.578125
/tmp/ipykernel_56/1666894758.py:86,2.0302734375
/tmp/ipykernel_56/1666894758.py:64,1.046875
/tmp/ipykernel_56/1666894758.py:82,0.4677734375
/tmp/ipykernel_56/1666894758.py:55,0.41796875
/tmp/ipykernel_56/1666894758.py:34,0.375
/tmp/ipykernel_56/1666894758.py:43,0.353515625



Top CPU Comsumers:
Function,CumTime_s
Q_Learning,3.611465083
<built-in method builtins.max>,1.188111286
Action,1.103416076
<built-in method builtins.round>,0.19200920200000002
<lambda>,0.166748386
<lambda>,0.13052586400000002
nxtPosition,0.125017185
<method 'copy' of 'dict' objects>,0.093963633
getReward,0.05617843300000001
isEndFunc,0.049594629



Takeaway: Accelerate this arithmetic-heavy update in hardware with fixed-point operators.

4. Verilog Accelerator Design

RTL Modules:

- **brent_kung_adder.v**: Fast parallel adder using prefix tree logic
- **brent_kung_subtractor.v**: Subtraction via 2's complement logic
- **wallace_multiplier.v**: Multiplier inferred via synthesis tools, maps to DSP/Wallace tree
- **barrel_shifter.v**: Fixed-point right-shift for scale correction
- **q_update_core.v**: Top-level module integrating the above to implement Bellman Q-update

Design Highlights:

- All values in fixed-point Q4.12 format (32-bit)
- Hardware pipeline designed to support one Q-update per cycle

- Fully modular and synthesis-ready structure (synthesis pending)

5. Key Steps & Milestones

Python Profiling

- cProfile used to isolate performance hotspots
- Confirmed arithmetic path as top contributor

PE Design (q_update_core.v)

- Computes: Q(s,a) <- (1-alpha)*Q(s,a) + alpha*(reward + gamma*maxQ)
- Integrates multiplier, subtractor, adder, shifter in fixed-point arithmetic

Integration (PyVerilator)

- Inputs: q_sa, reward, max_q, alpha, gamma
- Output: q_new
- Bit-level conversions done using helper functions in Python

6. Simulation & Verification

- Standalone testbench (q_update_tb.sv) to verify all operations
- Python test: Inputs fed to RTL, compared against floating-point Python baseline
- Verified numerical accuracy within 0.5% tolerance

Common Simulation Checks:

- Verify corner cases: reward=0, alpha=1.0, gamma=0
- Zero Q-values, high rewards
- Edge overflow/underflow

Problems faced while simulation and verification:

Metastability and Synchronization in co-simulation

- When co-simulating via bus model, mismatched handshake timing between Python and RTL leads to overruns or deadlock.

Floating-Point vs. Fixed-Point Discrepancies

- The Python “golden” uses float math, whereas your RTL uses 32-bit fixed. Discrepancies happened when writing testbench. Had to convert the values into fixed and several times.

Randomized Test Coverage Gaps

- Purely random action selection in a testbench never hit certain corner-case state transitions (holes right next to goal), leaving untested logic branches. This has been overlooked due to the available timing constraints for the project.

7. Functional Verification

- Python-to-Verilog interface tested using PyVerilator
- Sweep test: all 32-bit ranges explored with random tuples
- Assertions added to compare RTL vs software values

8. Synthesis

Synthesis has been attempted using OpenLane targeting an Artix-7 FPGA.

- Resource utilization:
  - LUTs: ~250
  - Flip-Flops: ~100
  - DSP slices: 1 (for multiplication)

Problems faced during synthesis:

Critical path violations:

I'm seeing warnings like "path delay = 12.3 ns exceeds clock period 10 ns" on the subpath mul1 → shift1 → add1 which is a critical path violation

Wallace-Tree Multiplier Reduction Tree

- A 16×16-bit multiplier built as a Wallace (or Dadda) tree has on the order of log3(W) reduction levels, each level doing a layer of full-adder logic. That entire tree is easily be 5–6 ns of logic alone, leaving too little margin for the rest of the datapath.

Brent-Kung Prefix Adder/Subtractor

- Although Brent-Kung is faster than a ripple-carry adder, it still has log_2(W) stages of "generate/propagate" logic plus a final sum stage. For 32 bits that's ≈8 prefix levels plus sum, which can be 2–3 ns of delay.

Attempted Strategies to overcome violations:

- **Inserted Pipeline Registers:** Broke the long chain into 2–3 stages. For example, register the result after each shift or multiplier.
- **Floorplan Critical Blocks:** Constrained placement so that the longest operations live close together physically.

- Used lint for static code analysis, checking code for syntax issues, coding style problems, poor practices, or potential logic errors before synthesis.

Placement violations:

Timing-Driven Placement Conflicts

- When the placement is done to optimize the critical multiply-shift-add chain for timing, it pushes logic apart to meet setup constraints only to create hold violations on the short paths. I can see "placement slack trade-offs" flagged in the timing report.

High-Fanout Net Trees

- Constants like one_fixed or the shared clock/reset signals fan out to every instantiation. I am getting a big buffer tree that adds skew to the signals.

Multiple DSP blocks.

- For performing 3 multiplications independently (for γ·maxQ, α·delta, and (1–α)·Q) to implement pipeline, ended up using too much space and created a giant hotspot that uses 30% of the design resources like power. If less than 3 multipliers are used, pipelining is not possible without introducing idle clocks.

Further optimization for power and area is possible for ASIC deployment.

- RTL timing analysis
- Resource estimation (LUTs, FFs, DSPs)
- Power and area optimization (post-synthesis)

9. Topics Learnt and Implemented

- HW/SW Partitioning in RL agents
- Fixed-point arithmetic and scaling in Verilog
- Brent-Kung and Wallace Tree microarchitectures
- PyVerilator co-simulation
- Performance profiling and bottleneck detection
- Integration of Python + RTL modules
- Q-learning logic and policy convergence

10. Next Steps & Future Work

- Complete synthesis for FPGA/ASIC targets
- Integrate on-chip BRAM for Q-table
- Explore adaptive epsilon decay logic in hardware
- Support larger grids (10x10) and Q(s,a) compression
- Automate Q-update invocation via AXI or DMA for SoC deployment